

NAME OF THE COURSE: OPERATING SYSTEMS

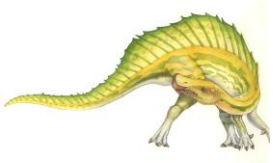
B.TECH V Semester

Course Code: 22PC1IT02

Credits:3

Text Book:
1.Operating System Principles, Abraham Silberchatz, Peter B. Galvin, Greg Gagne, 7th Edition, John Wiley

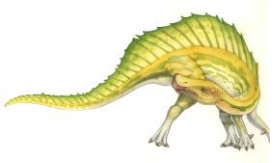
COURSE COORDINATOR NAME: K.SREENIVASA RAO





COURSE OBJECTIVES:

- 1. To understand operating system overview, Functions and services
- 2. To analyze process concept,inter process communication and synchronization
- 3. To understand memory management and file system interface
- 4. To understand secondary storage management
- 5. To understand protection and security





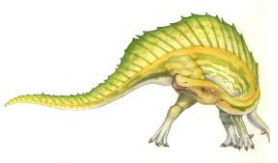
SYLLABUS

UNIT I:

- Operating system overview
- Process concept

UNIT II:

- Process management
- Process Synchronization





UNIT III:

- IPC
- Deadlocks

UNIT IV:

- Memory Management
- File Management

UNIT V:

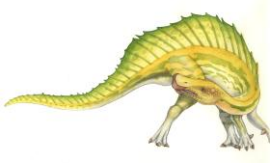
- Secondary storage
- Protection & security





Operating System Definition

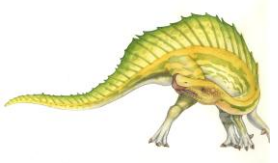
- Interface between user and hardware
- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls and coordinates use of hardware among various applications and users





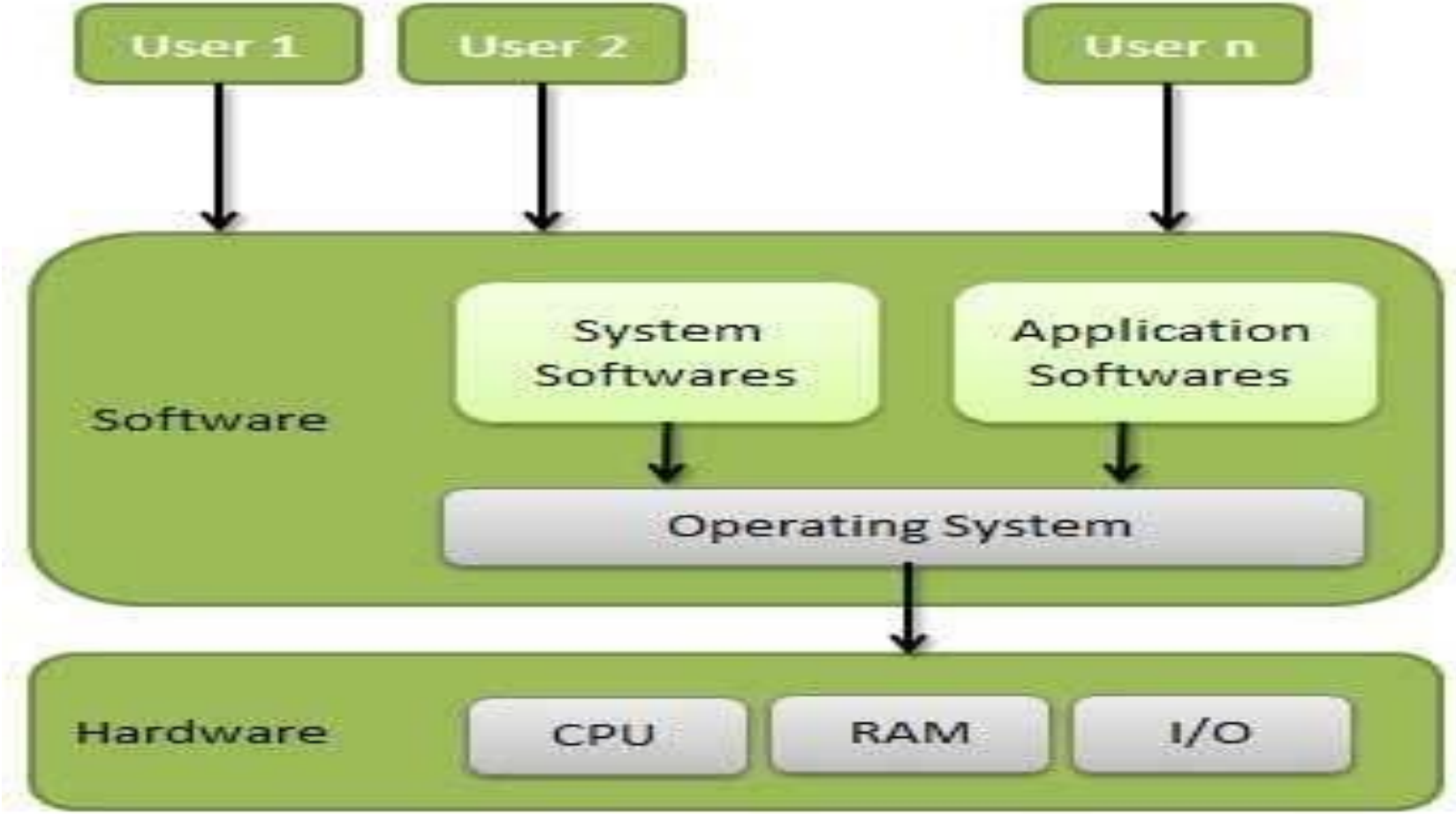
What is an Operating System?

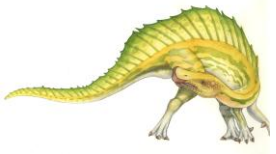
- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner





What is an Operating System?

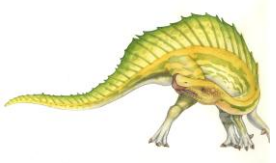






Examples of OS





Operating system

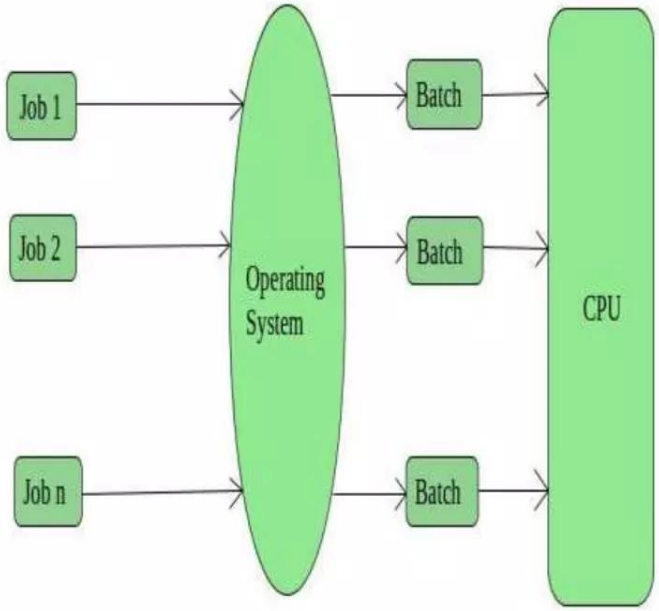
- An operating system (OS) is a **collection of programs** that facilitates execution of user programs and use of resources.
- There are **5 types** of operating system.
  - Batch processing operating system
  - Multiprogramming operating system
  - Time sharing operating system
  - Real time operating system
  - Distributed operating system



Batch Processing OS

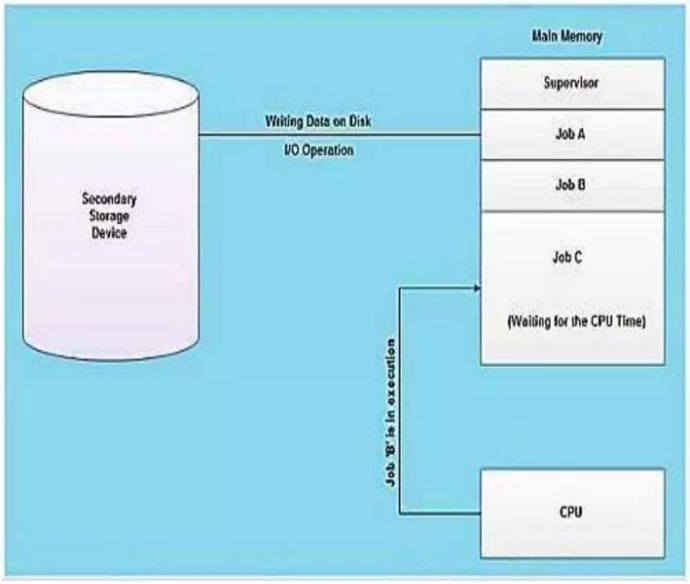
- They **do not interact** with the computer **directly**.
- There is an operator which takes similar jobs having the same requirement and group them into **batches.**
- Examples:

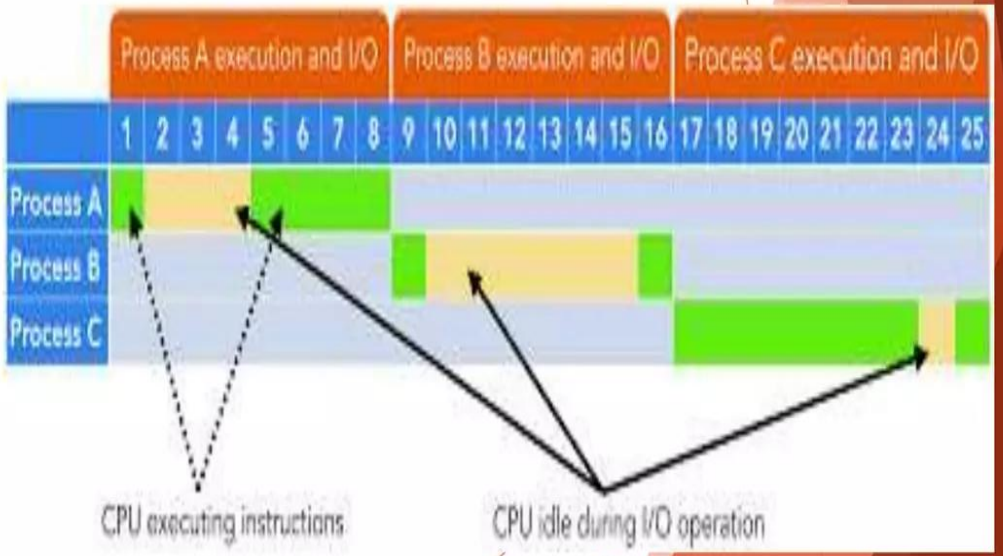
  Payroll System, Bank Statements, Daily Report, Research Segment, Billing System and etc.



Multiprogramming OS

- The multi-programming is interleaved execution of **multiple jobs** by the same Computer.
- when one program is waiting for I/O transfer; there is another program ready to utilize the CPU. So, it is possible for **several jobs** to share the time of the CPU.
- A simple process of multi-programming is shown in figure

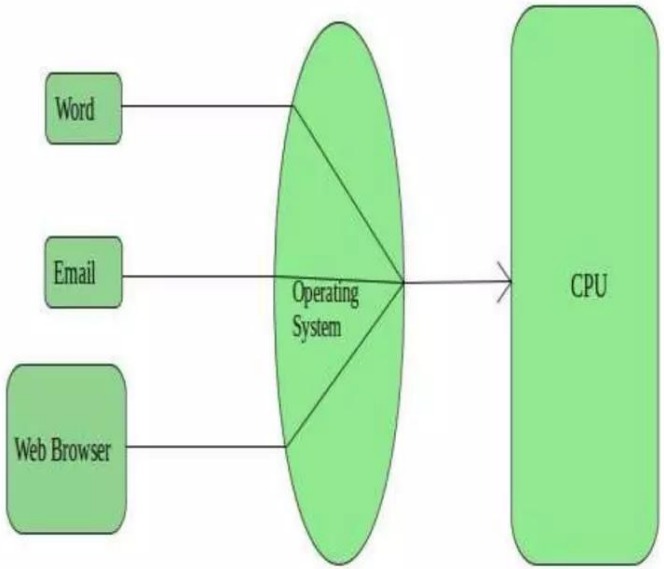




Time-Sharing Operating Systems

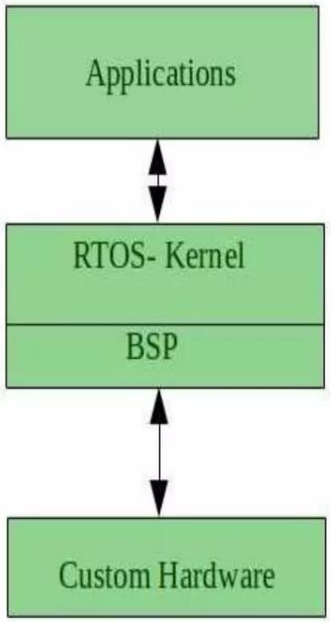
- Each task is given some **time to execute** so that all the tasks work smoothly.
- **Multitasking** Operating system
- The task can be from a single user or different users also.
- The time that each task gets to execute is called **Quantum.**
- After this time interval is over OS switches over to the next task.
- Examples :

  Multics, UNIX, et



Real time OS

- These types of OS's serve **real-time systems.**
- time interval to process and respond to inputs is **very small**. This time interval is called **response time**.
- Real-time systems are used when there are **time** requirements that are very **strict**
- Example: missile systems, air traffic control systems, robots, etc.

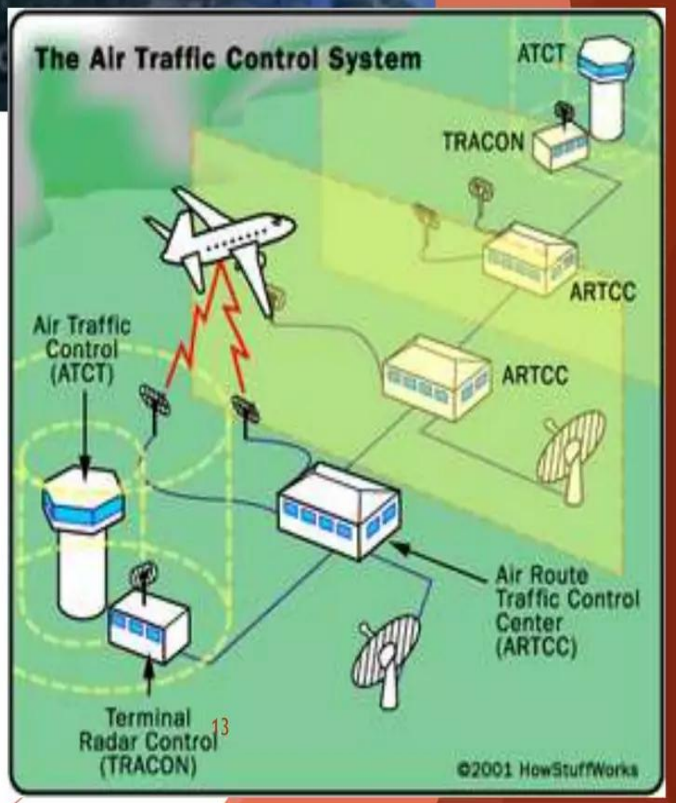


Air Traffic control system

- **Air traffic control (ATC)** is a service provided by ground-based controllers who direct aircraft on the ground and through controlled airspace and can provide advisory services to aircraft in non-controlled airspace.
- The primary purpose :

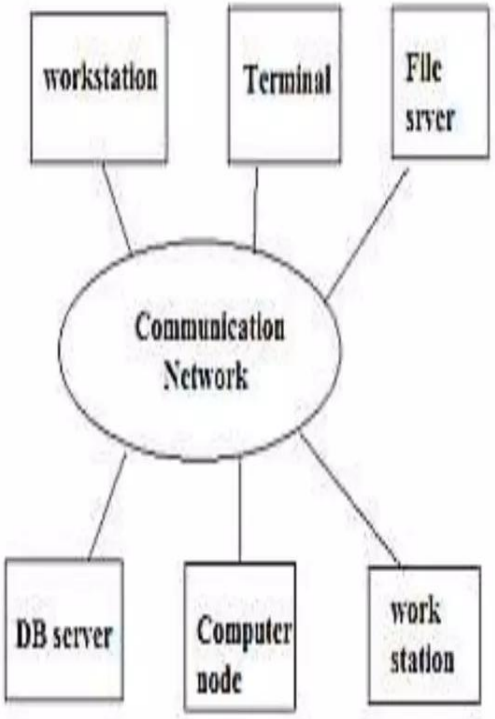
  to prevent collisions, organize and expedite the flow of traffic, and provide information and other **support for pilots** .
- ATC plays a security or defensive role or is operated by the **military.**





Distributing Operating System

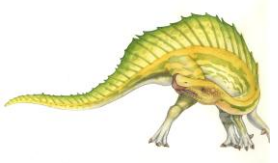
- A **collection of software** that manages computer hardware resources and provides common services for computer programs.
- Processors communicate with each other through various communication lines are known as **loosely coupled system**s or distributed systems.
- In a DOS the following occurs:
  - ✓ All software and hardware compounds are located **remotely.**
  - ✓ **Resource sharing**: Resources are managed by servers and clients use these resources.





Functions of an operating System.

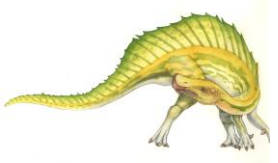
- Resource Manager
- Process Management
- Memory Management
- File Management
- Mass storage Management
- I/O Management
- Protection & Security





OS as a resource Manager

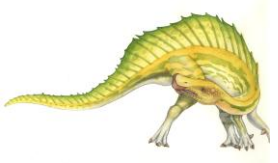
- Internally an **Operating System** acts as a **manager** of **resources** of the computer **system** such as processor, memory, files, and I/O device. In this role, the **operating system** keeps track of the status of each **resource**, and decides who gets a **resource**, for how long and when.





Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a *passive entity*, process is an *active entity*.
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources





Process Management Activities

The operating system is responsible for the following activities in connection with process management:

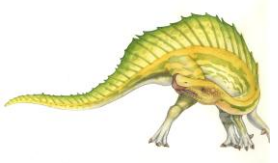
- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory Management

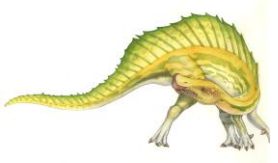
- All data in memory before and after processing
- All instructions in memory in order to execute
- Memory management determines what is in memory when
  - Optimizing CPU utilization and computer response to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed





File Management

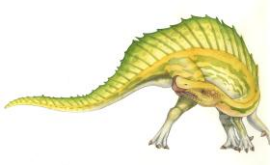
- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and dirs
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media





Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- OS activities
  - Free-space management
  - Storage allocation
  - Disk scheduling





I/O Management

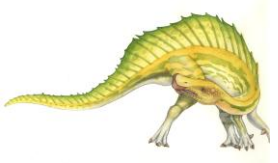
- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
  - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
  - General device-driver interface
  - Drivers for specific hardware devices





Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service

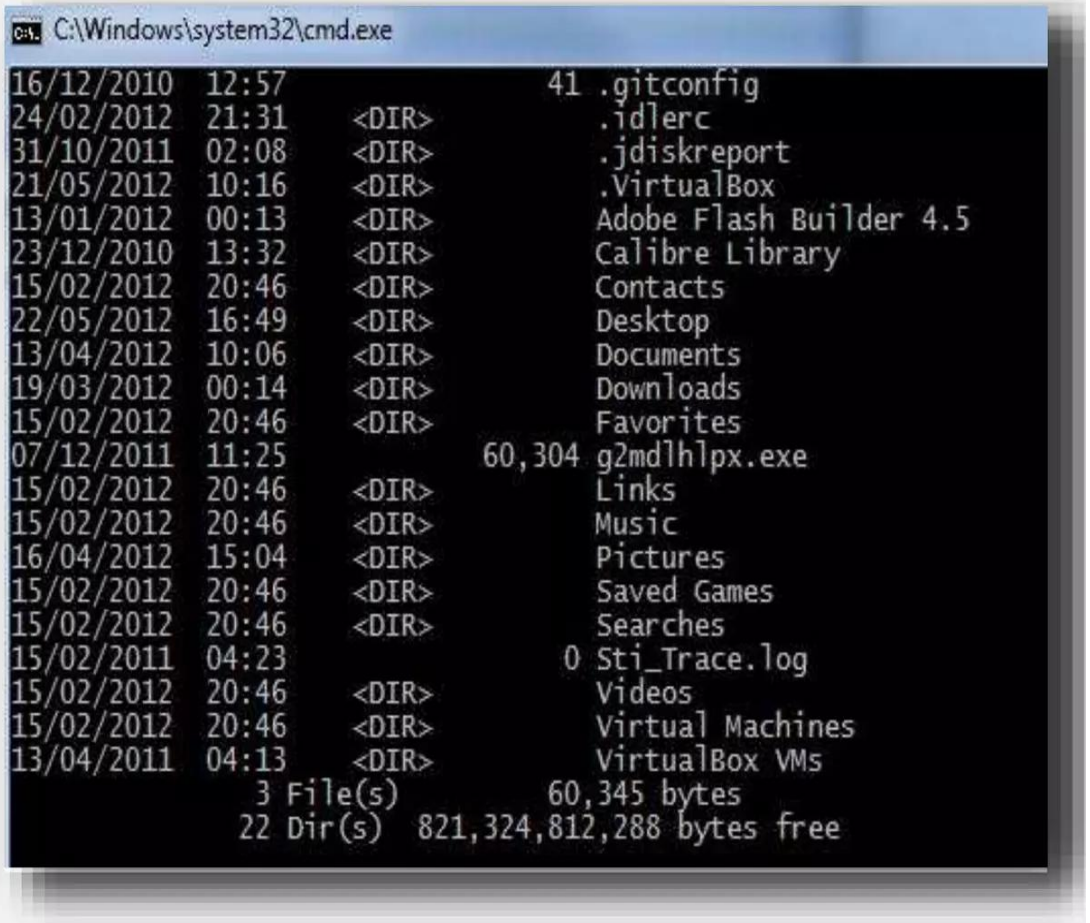




Operating System Services

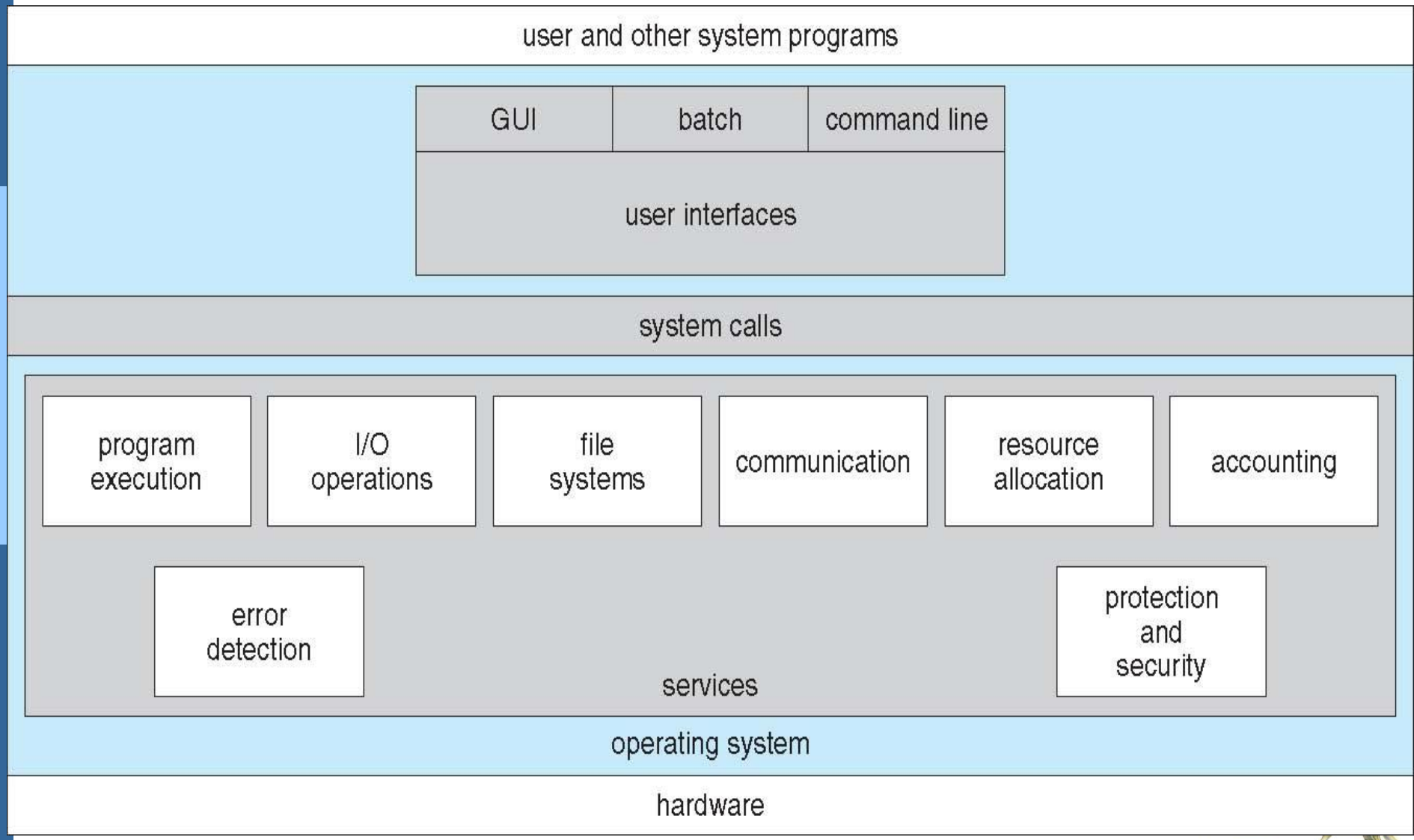
Services

- User interface
  - ✓ GUI
  - ✓ Batch
  - ✓ Command
- Program execution
- I/O operations
- File-system manipulation
- Communications
- Error detection





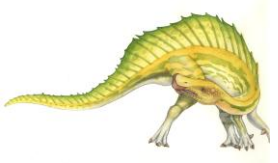
A View of Operating System Services





Operating System Services

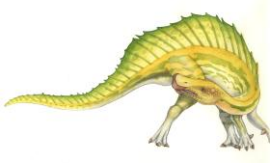
- One set of operating-system services provides functions that are helpful to the user:
  - User interface - Almost all operating systems have a user interface (UI)
    - Varies between Command-Line (CLI), Graphics User Interface (GUI), Batch
  - Program execution - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - I/O operations - A running program may require I/O, which may involve a file or an I/O device
  - File-system manipulation - The file system is of particular interest. Obviously, programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.





Operating System Services (Cont)

- One set of operating-system services provides functions that are helpful to the user (Cont):
  - Communications – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - Error detection – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system





Operating System Services (Cont)

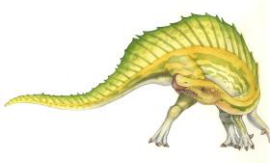
- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
    - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.





User Operating System Interface - GUI

- User-friendly desktop metaphor interface
  - Usually mouse, keyboard, and monitor
  - Icons represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI “command” shell
  - Apple Mac OS X as “Aqua” GUI interface with UNIX kernel underneath and shells available
  - Solaris is CLI with optional GUI interfaces (Java Desktop, KDE)



What are System Calls

- System Calls provide the Interface between a process and the Operating System.
- These calls are generally available as Assembly language instruction.
- System Calls can also be made directly through HLL programs for certain systems.
- UNIX System calls can be invoked directly from a C or C++ program.

Categories of System Calls

System calls can be grouped into five major categories as follows.

- Process control
- File management.
- Device management
- Information Maintenance and
- Communication.

Process control

Some system calls under process control are:

- End, abort
- Load, execute
- Create process, terminate process
- Get process, terminate process
- Wait for time
- Allocate and free memory

File management.

Some system calls under file management are:

- Create file, delete file
- Open , close
- Read, write, reposition.
- Get file attributes, set fil attributes

Device management

Some system calls under device management are:

- Request Device, release device
- Read, write, reposition.
- Get device attributes and set device attributes
- Logically attach or detach devices

Information Maintenance

Some system calls under information maintenance are:

- Get time or date, Set time of date
- Logically attach or detach devices
- Information maintenance
- Get system data, Set Systems data
- Get process, file of device attributes
- Set process, file or device attributes



Communication

- Create, delete communication connection.
- Send, receive messages
- Transfer status information
- Attach or detach remote devices.



Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



System Programs

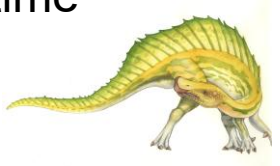
- Also known as system utilities
- Provide convenient environment for program development and execution
- Categories

1. File management
2. Status information
3. File modification
4. Programming language support
5. Program loading and execution
6. Communication
7. Application programs



Process Concept

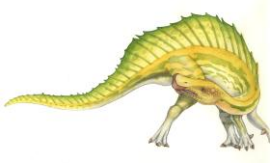
- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time





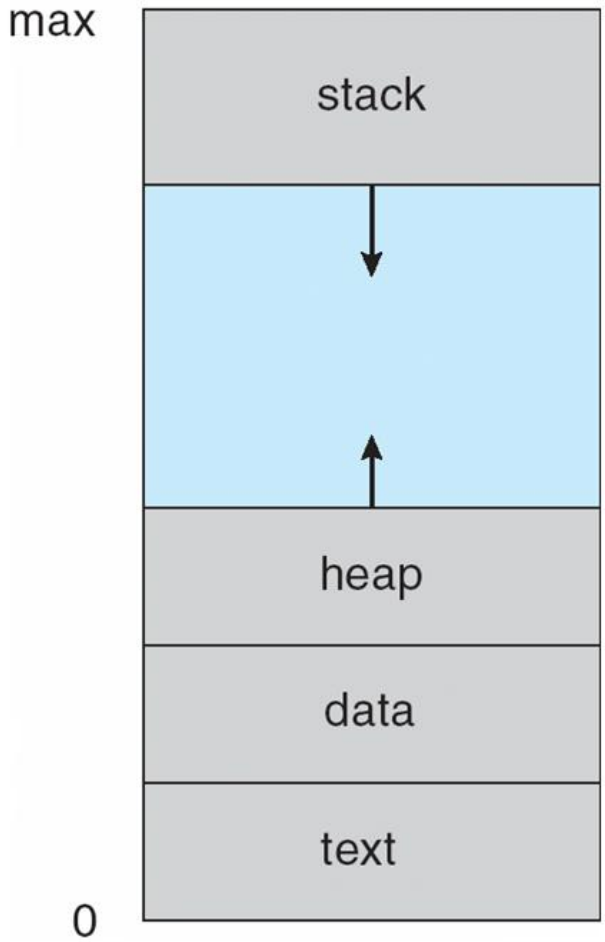
Process Concept (Cont.)

- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program





Process in Memory



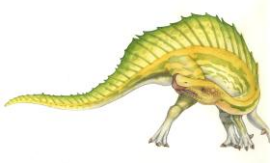
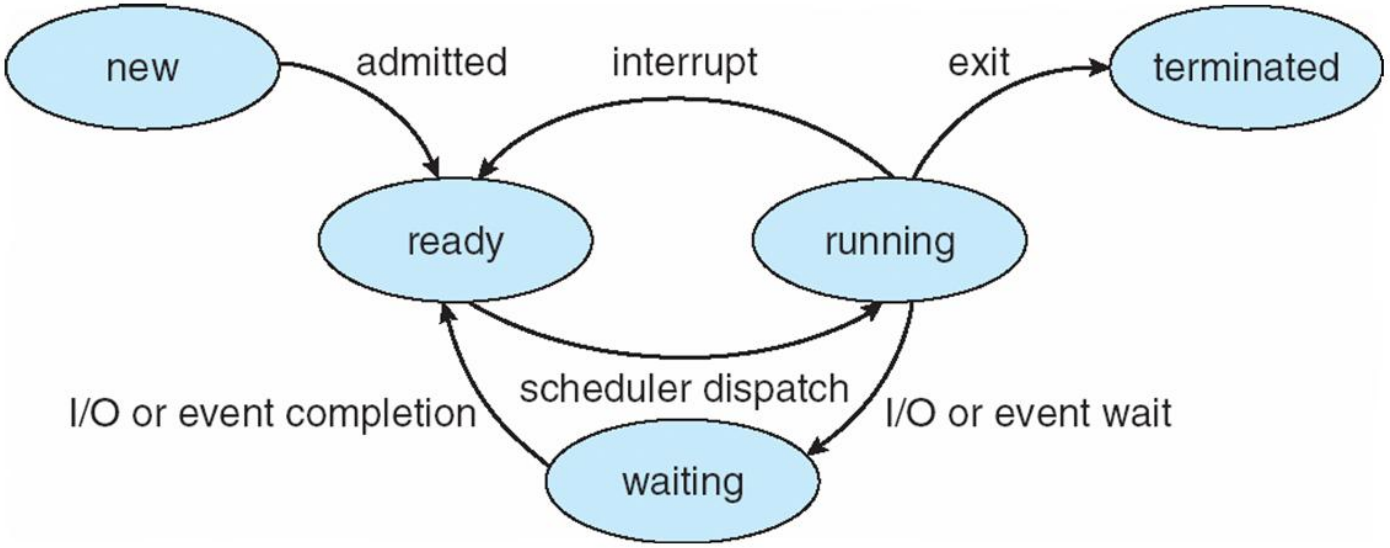




Diagram of Process State

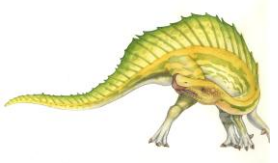






Process State

- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution



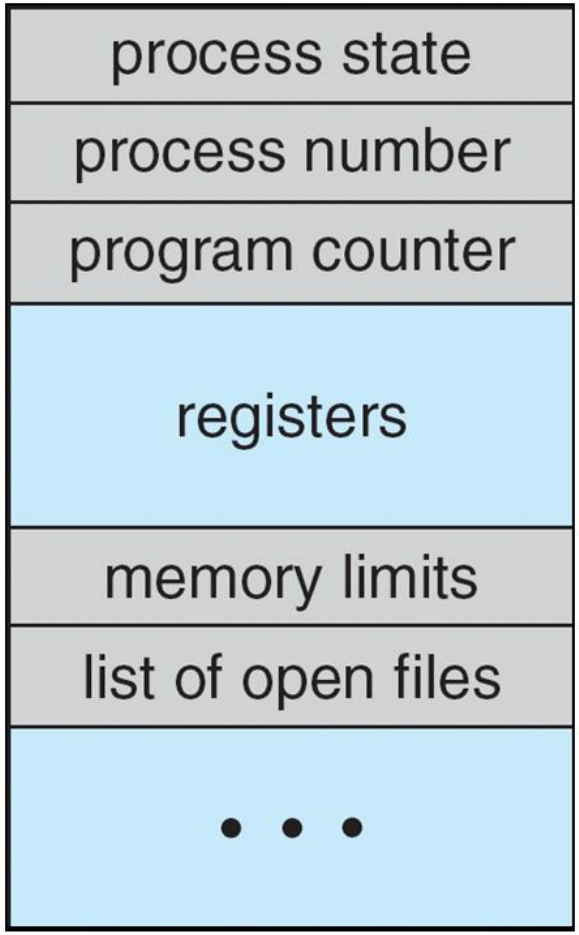


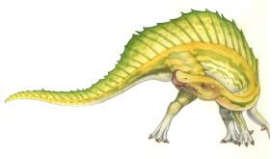
Process Control Block (PCB)

Information associated with each process

(also called **task control block**)

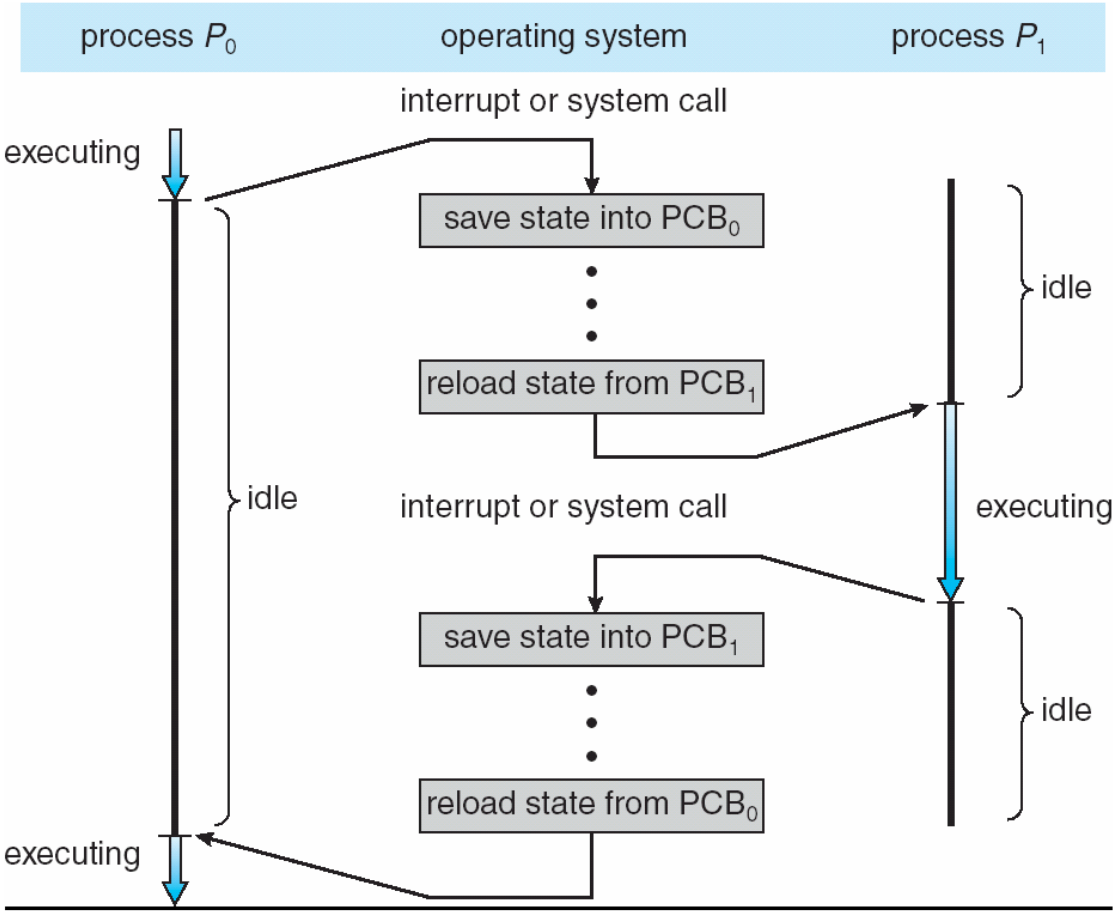
- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

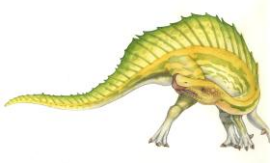






CPU Switch From Process to Process

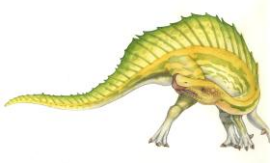






Threads

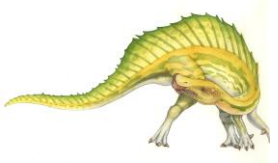
- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures





Multicore Programming (Cont.)

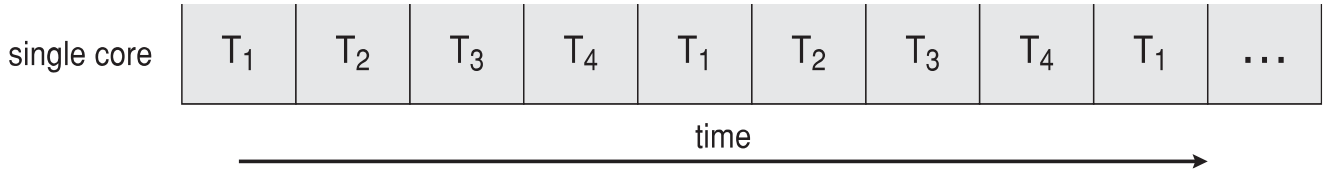
- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation
- As # of threads grows, so does architectural support for threading
  - CPUs have cores as well as ***hardware threads***
  - Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core



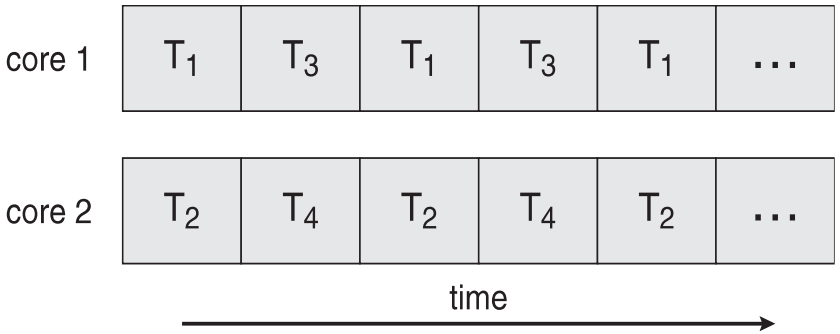


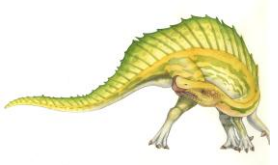
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



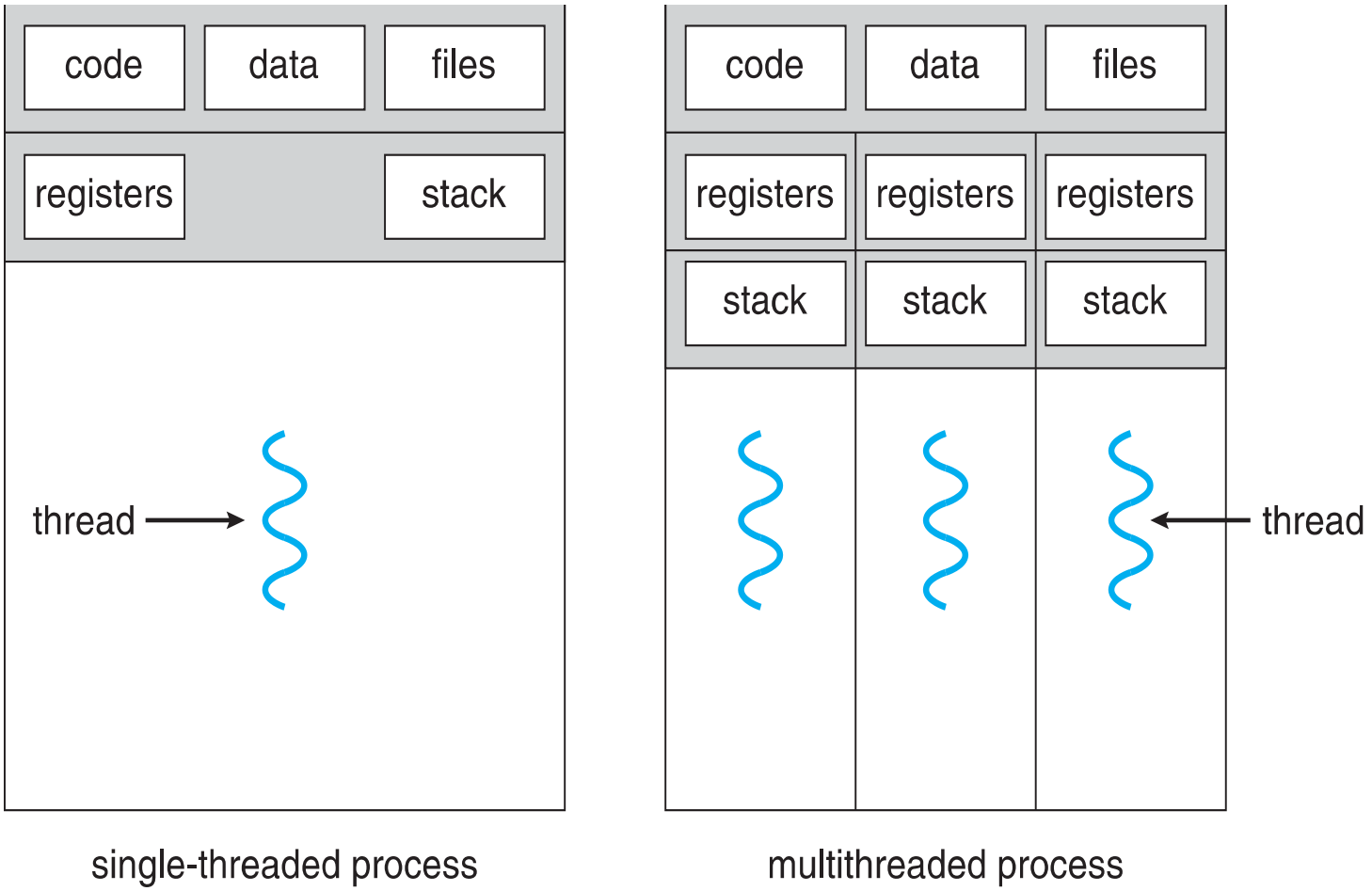
- **Parallelism on a multi-core system:**

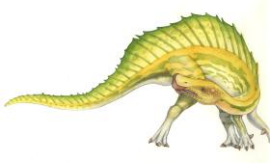






Single and Multithreaded Processes

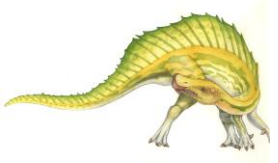






Multithreading Models

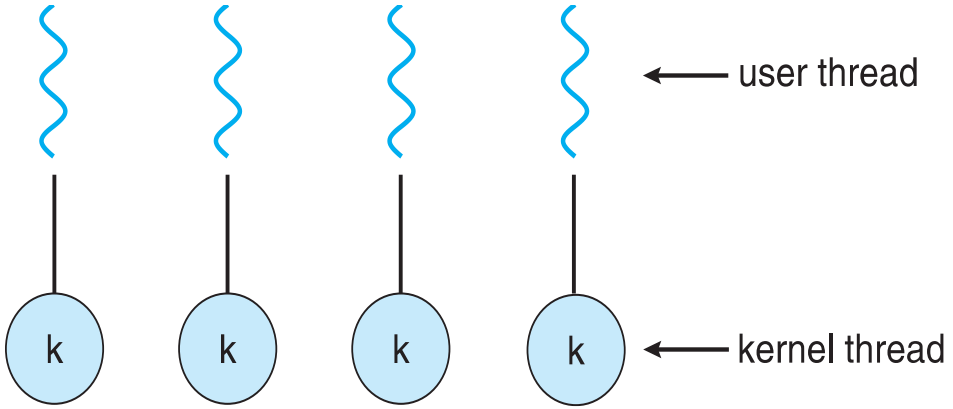
- Many-to-One
- One-to-One
- Many-to-Many

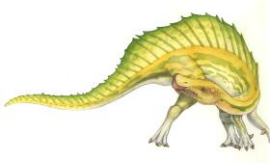




One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux
  - Solaris 9 and later

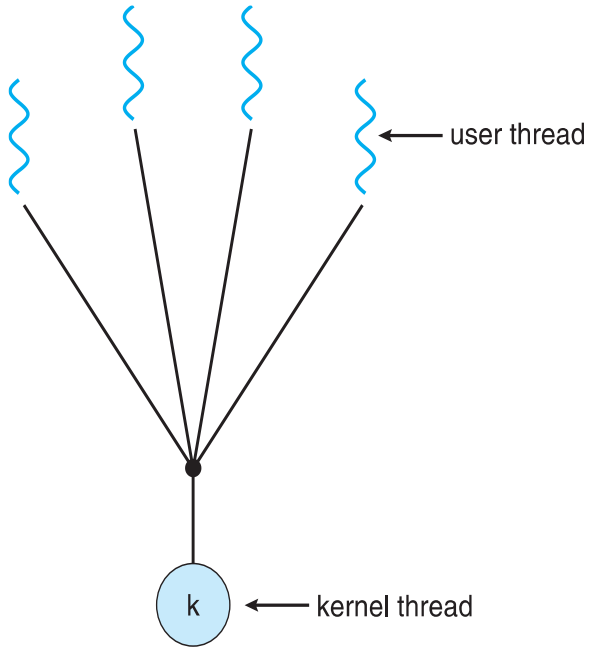


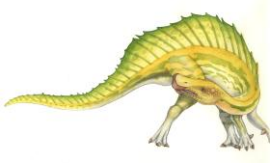




Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**







Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the *ThreadFiber* package

